

Secure File Transfer Protocol

Birfatehjrit Grewal

Networking Protocol

The client and server use a simple TCP implementation that I built on top of UDP to provide reliable data transfer. The protocol limits the clients outgoing packets with a set size sliding window for flow control. The retransmission is based on fast retransmit from TCP where three duplicate ACKs result in the retransmission of the lowest sequence number packet in the window. The default timeout retransmit is also implemented which will retransmit the lowest sequence number packet in the window.

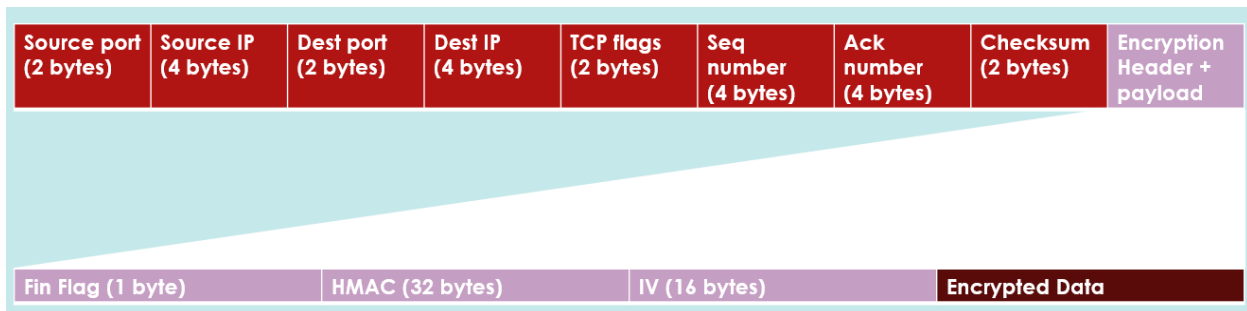
The TCP handshake was modified to include some additional information for the key exchange. The key exchange is explained in further details later. The server opens a half connection when it receives a packet with a SYN flag from the client. The server then returns a packet with both an ACK and SYN flag to the client. The state of the connection on the client's side changes to established once it receives the SYN from the server while the servers state remains as half open until it receives the next packet from the client.

The TCP sequence number is byte based and increased by the segment size of the file being transmitted. The sequence number is also offset by one due to the handshake. The TCP header used in this protocol is shown below. The header includes the source IP address, source port, destination IP address, destination port, TCP flags and a checksum.

Header Format

TCP Header – Red

Encryption Header – Pink



Key Exchange

The choice for the key exchange was based on the use case of this server. The key exchange uses a pre-shared key along with two nonce's to generate the session key for both encryption and the HMAC key for checking integrity. The session key is never transmitted but instead is derived on both sides using the nonce sent by the client along with SYN packet in the handshake and the servers nonce sent by the servers as a response to the SYN packet. The encryption key is a 128-bit long key used for AES-128 encryption and decryption. While the HMAC key is a 256-bit long key used to verify the integrity of the data being received.

Confidentiality

The transfer protocol uses AES-128 with CBC (Cipher Block Chaining) to encrypt each packet. The IV used during encryption is sent as a part of the encryption header of the packet. The iv along with the encryption key generated using the pre-shared key are used to decrypt the packet data.

Integrity

The Integrity of the packet is maintained using hash created using the HMAC key generated by HKDF with the pre-shared key. The hash output is computed using the key, iv, and encrypted bytes. The output is then sent as a part of the encryption header to the server. The server then computes the HMAC itself and compares the result with the HMAC in the packet.

Authentication

The authentication takes advantage of the pre-shared key to authenticate the client and server during the TCP handshake. The pre-shared key is only known to the client and the server due to it being a random 32 bytes exchanged through physical transfer of the text file (for instance a usb). The handshake uses the key and the nonce_c and nonce_s to authenticate the client and the server.

Future Additions

Public key infrastructure: To handle key exchange

User differentiation: User credentials

Packet Filtering: Filter out packet outside of expected IP subnets

File retrieval: Request a file from the server